CI NICARAGUA · 2026

TEMPORADA XIV

DOCUMENTO DE LECTURA

09

// Unidad II · Desarrollo Backend con Java

UNIR TABLAS CON JOIN

en SQL Server, MySQL y SQLite



EN ESTA LECTURA

01 ¿Por qué unir tablas?

02 Las relaciones del proyecto

03 Anatomía de un JOIN

04 INNER JOIN

05 LEFT, RIGHT y FULL JOIN

06 Varios JOIN y funciones de grupo

07 Los tres motores, lado a lado

08 El ORM y los JOIN

ELABORADO POR

MARVIN RAFAEL MEZA PINEDA

Facilitador · Centro Nacional de Innovación y Tecnología
Francisco "El Chele" Moreno

Fundamentos de Desarrollo Backend con Java

Noviembre 2026

CAPÍTULO 01

¿POR QUÉ UNIR TABLAS?

> Una base de datos relacional guarda cada dato en un solo lugar y conecta las tablas con claves. El JOIN es la instrucción de SQL que vuelve a juntar esa información cuando la necesitamos.

Esta lectura acompaña la presentación **Relaciones, JOINs y ORM vs SQL** y usa el código de la rama `clase-springboot-relaciones`: las entidades `Producto`, `Categoria` y `Cliente`, sus relaciones y los datos de ejemplo que carga `DatosIniciales.java`. Todo lo que aquí se muestra se puede ejecutar en los tres motores que soporta el proyecto: **SQL Server**, **MySQL** y **SQLite**.

El problema de guardar todo junto

Imagina que cada producto guardara escrito el nombre y la descripción de su categoría:

producto	precio	categoria	desc_categoria
Laptop	850	Electronica	Computadoras, celulares…
Audifonos	45	Electronica	Computadoras, celulares…
Balon de futbol	25	Deportes	Articulos deportivos…
Pesas 5 kg	30	Deportes	Articulos deportivos…

Funciona, pero tiene tres problemas. **Repite datos**: el mismo texto aparece en cada producto. **Es difícil de mantener**: si "Electronica" pasa a llamarse "Tecnología", hay que cambiar todas sus filas, y si se olvida una, la información queda inconsistente. Y **no protege nada**: cualquiera puede escribir "Electrónica", "electronica" o "Elctronica" como si fueran categorías distintas.

La solución: separar y relacionar

Nuestro proyecto guarda las categorías en su propia tabla. Cada producto solo guarda un número, `categoria_id`, que apunta al `id` de su categoría. A esa columna se le llama **clave foránea** (*foreign key*, FK), y a este proceso de separar para no repetir, **normalización**.

`productos`

id	nombre	precio	categoria_id
1	Laptop	850	1
2	Audifonos	45	1
4	Balon de futbol	25	4

`categorias`

id	nombre	descripcion
1	Electronica	Computadoras, celulares y accesorios
4	Deportes	Articulos deportivos y fitness

Ahora el nombre de cada categoría vive en una sola fila. Pero surge una pregunta: si la tabla `productos` solo tiene números, ¿cómo mostramos "Laptop — Electronica"? Con un **JOIN**.

> **LA LISTA DE ASISTENCIA Y EL DIRECTORIO**
>
> Piensa en una lista de asistencia que solo tiene números de carné, y en un directorio que dice a quién pertenece cada carné. Ninguno de los dos papeles, por sí solo, te dice *quién* vino a clase. Pero si pones uno al lado del otro y emparejas los números, obtienes la lista con nombres. Eso es exactamente un JOIN: emparejar las filas de dos tablas usando una columna en común.

Por qué los JOIN son tan importantes

- **Sin datos repetidos:** cada dato vive en un solo lugar.
- **Un cambio, un lugar:** renombrar una categoría es actualizar una fila.
- **Integridad:** la clave foránea impide apuntar a una categoría que no existe (según el motor, como veremos en el capítulo 7).
- **Consultas reales:** casi todo reporte útil cruza tablas: productos por categoría, clientes por sus favoritos, ventas por vendedor.

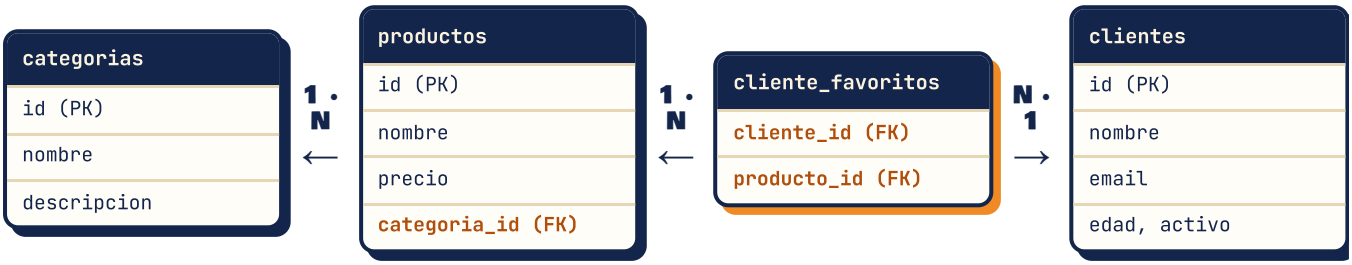
Por eso el JOIN es de lo primero que se pregunta en una entrevista técnica de backend, y por eso lo vamos a dominar en los tres motores.

CAPÍTULO 02

LAS RELACIONES DEL PROYECTO

> En la clase agregamos dos relaciones: muchos productos pertenecen a una categoría, y muchos clientes tienen muchos productos favoritos. Hibernate creó las tablas y las claves foráneas a partir de las anotaciones.

El diagrama de tablas



Las columnas en naranja son claves foráneas. Observa que la flecha sale siempre de la tabla que tiene la clave foránea y apunta a la tabla referenciada.

Relación uno a muchos: Producto → Categoría

Un producto tiene **una** categoría y una categoría tiene **muchos** productos. Basta con una columna en la tabla del lado "muchos":

Producto.java (resumido)

```java
@ManyToOne                          // muchos productos → una categoría
@JoinColumn(name = "categoria_id")  // nombre de la columna FK
private Categoria categoria;
```

`Producto` es el **dueño** de la relación porque en su tabla vive la clave foránea. `Categoria` tiene el lado inverso, `@OneToMany(mappedBy = "categoria")`, que no crea ninguna columna y lleva `@JsonIgnore` para que el JSON no entre en un ciclo infinito.

Relación muchos a muchos: Cliente ↔ Producto

Un cliente puede tener muchos favoritos y un producto puede ser favorito de muchos clientes. Eso no cabe en una columna: se necesita una **tabla intermedia** con una fila por cada pareja cliente–producto.

Cliente.java (resumido)

```java
@ManyToMany
@JoinTable(name = "cliente_favoritos",
    joinColumns        = @JoinColumn(name = "cliente_id"),
    inverseJoinColumns = @JoinColumn(name = "producto_id"))
private Set<Producto> favoritos;
```

Los datos de ejemplo

Todos los resultados de esta lectura salen de estos datos. Fíjate en los **casos especiales** (resaltados): son los que marcan la diferencia entre un tipo de JOIN y otro.

producto	precio	categoría
Laptop	850	Electronica
Audifonos	45	Electronica
Licuadora	60	Hogar
Balon de futbol	25	Deportes
Pesas 5 kg	30	Deportes
Clean Code	40	Libros
Producto sin categoria	10	NULL

cliente	favoritos
Ana Lopez	Laptop, Audifonos, Clean Code
Carlos Perez	Balon de futbol, Pesas 5 kg
Maria Gonzalez	Laptop, Licuadora
Lucia Ramirez	Clean Code
Jose Martinez	(ninguno)

Además, la categoría **Ropa** existe pero no tiene productos. Con una base nueva, los id de las categorías son: Electronica 1, Hogar 2, Ropa 3, Deportes 4 y Libros 5.

CAPÍTULO 03

ANATOMÍA DE UN JOIN

> Todo JOIN responde dos preguntas: qué tablas se unen y con qué regla se emparejan sus filas. La regla va después de la palabra ON.

estructura general · igual en los tres motores

```
SELECT columnas                      -- qué quiero ver
FROM   tabla_a a                     -- tabla de la izquierda (con alias "a")
JOIN   tabla_b b ON b.id = a.b_id     -- tabla de la derecha y regla de unión
WHERE  condición                     -- filtra el resultado ya unido (opcional)
ORDER BY columna;                    -- ordena (opcional)
```

Las piezas, una por una

Pieza	Qué hace
`FROM tabla_a`	La tabla de la **izquierda**. Importa en LEFT y RIGHT JOIN.
`JOIN tabla_b`	La tabla de la **derecha**. `JOIN` a secas significa `INNER JOIN`.
Alias (`p` , `c`)	Nombres cortos para no repetir el nombre completo de la tabla. Se escriben después del nombre: `productos p` .
`ON`	La regla para emparejar: normalmente *clave foránea = clave primaria*.
`tabla.columna`	Cuando dos tablas tienen una columna con el mismo nombre (`nombre` , `id`), hay que indicar de cuál es: `p.nombre` o `c.nombre` .
`AS`	Cambia el título de una columna en el resultado: `c.nombre AS categoria` .

ON NO ES LO MISMO QUE WHERE

`ON` dice *cómo* se emparejan las filas; `WHERE` filtra el resultado *después* de unirlas. En un INNER JOIN da igual dónde pongas una condición extra, pero en un LEFT JOIN no: una condición sobre la tabla de la derecha puesta en el `WHERE` elimina las filas que tenían `NULL` y convierte, sin querer, tu LEFT JOIN en un INNER JOIN.

Los cuatro tipos de JOIN

Si representamos cada tabla como un círculo, la parte amarilla es lo que devuelve cada tipo de JOIN (A es la tabla del FROM y B la del JOIN):



Existe además el **CROSS JOIN**, que combina cada fila de A con cada fila de B sin ninguna regla. Con 7 productos y 5 categorías devuelve 35 filas, casi todas sin sentido. Si olvidas el `ON` o lo escribes mal, puedes obtener un efecto parecido: muchas más filas de las esperadas es la señal de alarma.

Un JOIN puede devolver más filas que la tabla original

Hay 5 clientes, pero `clientes JOIN cliente_favoritos` devuelve 8 filas: una por cada favorito. Ana, con 3 favoritos, aparece 3 veces. Cuando queremos una sola fila por cliente usamos `DISTINCT` o `GROUP BY`, como veremos en el capítulo 6.

CAPÍTULO 04

INNER JOIN

> INNER JOIN devuelve solo las filas que encuentran pareja en las dos tablas. Es el JOIN más usado y se escribe exactamente igual en SQL Server, MySQL y SQLite.

Productos con el nombre de su categoría

SQL Server · MySQL · SQLite

```
SELECT p.nombre AS producto, p.precio, c.nombre AS categoria
FROM productos p
INNER JOIN categorias c ON c.id = p.categoria_id
ORDER BY p.precio;
```

RESULTADO · 6 FILAS

producto	precio	categoria
Balon de futbol	25	Deportes
Pesas 5 kg	30	Deportes
Clean Code	40	Libros
Audifonos	45	Electronica
Licuadora	60	Hogar
Laptop	850	Electronica

Hay 7 productos, pero el resultado tiene 6. ¿Dónde quedó "Producto sin categoria"? Su `categoria_id` es `NULL`, y `NULL` no es igual a nada, ni siquiera a otro `NULL`: no encuentra pareja y el INNER JOIN lo descarta. Lo mismo ocurre con Ropa, que no tiene productos.

JOIN con filtro

Se puede filtrar por columnas de cualquiera de las dos tablas. Esta consulta corresponde al método `buscarPorCategoriaYPrecio` del proyecto:

productos de Electronica entre 10 y 100

```
SELECT p.nombre, p.precio
FROM productos p
JOIN categorias c ON c.id = p.categoria_id
WHERE c.id = 1
  AND p.precio BETWEEN 10 AND 100
ORDER BY p.precio;                -- resultado: Audifonos 45
```

Clientes con sus favoritos

Para pasar de clientes a productos hay que atravesar la tabla intermedia, así que se necesitan dos JOIN:

SQL Server · MySQL · SQLite

```
SELECT c.nombre AS cliente, p.nombre AS favorito
FROM clientes c
INNER JOIN cliente_favoritos cf ON cf.cliente_id = c.id
INNER JOIN productos p          ON p.id = cf.producto_id;
```

El resultado tiene **8 filas** (Ana 3, Carlos 2, María 2 y Lucía 1). José Martínez no aparece porque no tiene ninguna fila en `cliente_favoritos`.

EL NOMBRE DE LAS COLUMNAS EN EL RESULTADO

Si quieres un título con espacios, como “Nombre Cliente”, cada motor tiene su forma preferida de escribirlo: `[Nombre Cliente]` en SQL Server, `` `Nombre Cliente` `` en MySQL y `"Nombre Cliente"` en SQLite. Las comillas dobles también funcionan en SQL Server (opción `QUOTED_IDENTIFIER`, activada por defecto) y en SQLite. En MySQL las comillas dobles funcionan para los alias, pero normalmente se leen como texto.

CAPÍTULO 05

LEFT, RIGHT Y FULL JOIN

> Los OUTER JOIN conservan las filas que no tienen pareja y rellenan con NULL las columnas que faltan. Son la herramienta para encontrar lo que falta: categorías vacías, clientes sin compras, productos sin categoría.

LEFT JOIN: todas las categorías

SQL Server · MySQL · SQLite

```
SELECT c.nombre AS categoria, p.nombre AS producto
FROM categorias c
LEFT JOIN productos p ON p.categoria_id = c.id
ORDER BY c.id;
```

RESULTADO · 7 FILAS

categoria	producto
Electronica	Laptop
Electronica	Audifonos
Hogar	Licuadora
Ropa	NULL
Deportes	Balon de futbol
Deportes	Pesas 5 kg
Libros	Clean Code

Ahora la tabla de la izquierda es `categorias`, y LEFT JOIN garantiza que todas aparecen. Ropa no tiene productos, así que su columna `producto` viene en `NULL`.

EL TRUCO DEL IS NULL

Para quedarte solo con lo que *no* tiene pareja, agrega `WHERE p.id IS NULL` al LEFT JOIN anterior: el resultado es únicamente **Ropa**. Con la misma idea encuentras los clientes sin favoritos (`clientes LEFT JOIN cliente_favoritos … WHERE cf.cliente_id IS NULL` devuelve a José Martínez). Ojo: se escribe `IS NULL`, nunca `= NULL`.

RIGHT JOIN: lo mismo, al revés

`productos p RIGHT JOIN categorias c` conserva todas las categorías, igual que el LEFT JOIN anterior con las tablas en el otro orden. En la práctica casi no se usa: se invierten las tablas y se escribe LEFT JOIN, que es más fácil de leer.

FULL OUTER JOIN: todo de los dos lados

Conserva las filas sin pareja de ambas tablas. Aquí la diferencia entre motores es real:

SQL Server · SQLite (3.39 o superior)

```
SELECT p.nombre AS producto, c.nombre AS categoria
FROM productos p
FULL OUTER JOIN categorias c ON c.id = p.categoria_id;
```

MySQL · no tiene FULL JOIN: se simula con UNION

```
SELECT p.nombre AS producto, c.nombre AS categoria
FROM productos p LEFT JOIN categorias c ON c.id = p.categoria_id
UNION
SELECT p.nombre, c.nombre
FROM productos p RIGHT JOIN categorias c ON c.id = p.categoria_id;
```

En los dos casos el resultado tiene **8 filas**: los 6 productos con categoría, "Producto sin categoria" con categoría `NULL` y Ropa con producto `NULL`. `UNION` junta los dos resultados y elimina las filas repetidas.

¿Cuál uso?

Necesito…	Uso
Solo lo que tiene pareja	`INNER JOIN`
Todo de una tabla, tenga o no pareja	`LEFT JOIN` (con esa tabla en el FROM)
Lo que *no* tiene pareja	`LEFT JOIN … WHERE b.id IS NULL`
Todo de las dos tablas	`FULL OUTER JOIN`, o LEFT + UNION + RIGHT en MySQL

CAPÍTULO 06

VARIOS JOIN Y FUNCIONES DE GRUPO

> Los reportes de verdad combinan JOIN con GROUP BY y funciones como COUNT, SUM y AVG. Así se responde "cuántos productos tiene cada categoría" o "qué clientes tienen favoritos de Electrónica".

Reporte: productos por categoría

Esta es la consulta del endpoint `GET /api/categorias/resumen`:

SQL Server · MySQL · SQLite

```sql
SELECT c.nombre,
       COUNT(p.id)   AS total,
       AVG(p.precio) AS promedio
FROM categorias c
LEFT JOIN productos p ON p.categoria_id = c.id
GROUP BY c.id, c.nombre
ORDER BY total DESC;
```

RESULTADO · 5 FILAS

nombre	total	promedio
Electronica	2	447.5
Deportes	2	27.5
Hogar	1	60.0
Libros	1	40.0
Ropa	0	NULL

`GROUP BY` junta las filas del JOIN por categoría y las funciones calculan un valor por grupo. Usamos LEFT JOIN para que Ropa aparezca con total 0. El orden entre categorías empatadas puede variar de un motor a otro.

COUNT(*) ENGAÑA

`COUNT(*)` cuenta filas, y Ropa tiene una fila en el LEFT JOIN (con producto `NULL`), así que daría 1. `COUNT(p.id)` cuenta solo los valores que no son `NULL` y da 0, que es lo correcto. Con LEFT JOIN, cuenta siempre una columna de la tabla de la derecha.

Tres JOIN seguidos: clientes con favoritos de una categoría

Para ir de clientes a categorías hay que recorrer el camino completo: clientes → cliente_favoritos → productos → categorias.

SQL Server · MySQL · SQLite

```sql
SELECT DISTINCT cl.nombre
FROM clientes cl
JOIN cliente_favoritos cf ON cf.cliente_id = cl.id
JOIN productos p          ON p.id = cf.producto_id
JOIN categorias c         ON c.id = p.categoria_id
WHERE c.id = 1;             -- Electronica → Ana Lopez, Maria Gonzalez
```

Ana tiene dos favoritos de Electrónica (Laptop y Audífonos); sin `DISTINCT` aparecería dos veces. Es la consulta del endpoint `GET /api/clientes/por-categoria-favorita/1`.

Total de los favoritos de cada cliente

SQL Server · MySQL · SQLite

```sql
SELECT cl.nombre, SUM(p.precio) AS total
FROM clientes cl
JOIN cliente_favoritos cf ON cf.cliente_id = cl.id
JOIN productos p          ON p.id = cf.producto_id
GROUP BY cl.id, cl.nombre;   -- Ana 935 · Carlos 55 · Maria 910 · Lucia 40
```

Regla de oro de `GROUP BY`: toda columna del `SELECT` que no esté dentro de una función de grupo debe aparecer en el `GROUP BY`. SQL Server la exige siempre; MySQL la exige con su configuración por defecto; SQLite es más permisivo, pero no conviene aprovecharlo.

CAPÍTULO 07

LOS TRES MOTORES, LADO A LADO

> Los JOIN que vimos se escriben igual en SQL Server, MySQL y SQLite. Lo que cambia está alrededor: cómo se elige la base, cómo se limitan filas, cómo se comparan textos y si se respetan las claves foráneas.

Tabla comparativa

Tema	SQL Server	MySQL	SQLite
Qué es	Servidor (puerto 1433)	Servidor (puerto 3306)	Un archivo: `demo.db`
Elegir la base	`USE crud_demo;` + `GO`	`USE crud_demo;`	Abrir el archivo: `sqlite3 demo.db`
Nombres con espacios	`[Nombre Cliente]`	`` `Nombre Cliente` ``	`"Nombre Cliente"`
Limitar filas	`SELECT TOP 3 …`	`… LIMIT 3`	`… LIMIT 3`
Unir textos	`CONCAT(a, b)` o `a + b`	`CONCAT(a, b)`	`a || b`
= con mayúsculas	No distingue (collation por defecto)	No distingue (collation por defecto)	Sí distingue
Claves foráneas	Siempre se cumplen	Siempre se cumplen (InnoDB)	Apagadas por defecto
RIGHT JOIN	Sí	Sí	Desde la versión 3.39
FULL OUTER JOIN	Sí	No: LEFT + UNION + RIGHT	Desde la versión 3.39
Booleanos	`bit` (0 / 1)	`bit` o `tinyint` (0 / 1)	Entero (0 / 1)
Herramienta	SSMS o Azure Data Studio	MySQL Workbench	sqlite3 o DB Browser for SQLite
Dialecto de Hibernate	`SQLServerDialect`	`MySQLDialect`	`SQLiteDialect`

La misma consulta en los tres motores

Los tres productos más caros con su categoría. Solo cambia la forma de pedir “los primeros 3”:

SQL Server

```
USE crud_demo;
GO
SELECT TOP 3 p.nombre AS [Producto], c.nombre AS [Categoria]
FROM productos p
INNER JOIN categorias c ON c.id = p.categoria_id
ORDER BY p.precio DESC;
```

MySQL

```mysql
USE crud_demo;
SELECT p.nombre AS `Producto`, c.nombre AS `Categoria`
FROM productos p
INNER JOIN categorias c ON c.id = p.categoria_id
ORDER BY p.precio DESC
LIMIT 3;
```

SQLite · en la terminal: sqlite3 demo.db

```sqlite
.headers on
.mode column
SELECT p.nombre AS "Producto", c.nombre AS "Categoria"
FROM productos p
INNER JOIN categorias c ON c.id = p.categoria_id
ORDER BY p.precio DESC
LIMIT 3;
```

Resultado en los tres: Laptop (Electronica), Licuadora (Hogar) y Audifonos (Electronica). En SQLite, `.headers on` y `.mode column` no son SQL: son comandos de la consola para ver el resultado como tabla.

Diferencia 1: las mayúsculas

la categoría se guardó como “Deportes”

```mysql
SELECT p.nombre FROM productos p
JOIN categorias c ON c.id = p.categoria_id
WHERE c.nombre = 'deportes';
```

SQL Server y MySQL devuelven 2 filas (Balon de futbol y Pesas 5 kg), porque su collation por defecto no distingue mayúsculas. SQLite devuelve **0 filas**: para él, `'deportes'` y `'Deportes'` son textos distintos. Por eso el JPQL del proyecto compara con `LOWER(c.nombre) = LOWER(:nombre)`: así da lo mismo en los tres motores.

Diferencia 2: la integridad referencial

SQL directo, saltándose la API

```mysql
DELETE FROM categorias WHERE id = 1;   -- Electronica tiene productos
```

- **SQL Server** rechaza el borrado con el error 547 (“The DELETE statement conflicted with the REFERENCE constraint…”).
- **MySQL** lo rechaza con el error 1451 (“Cannot delete or update a parent row: a foreign key constraint fails”).
- **SQLite** trae las claves foráneas apagadas: borra la categoría y los productos quedan apuntando a un id que ya no existe. Se activan, en cada conexión, con `PRAGMA foreign_keys = ON;`, y solo protegen si la tabla se creó con la restricción.

Por eso `CategoriaController` no confía en la base: antes de borrar revisa con `existsByCategoriaId` y responde **409 Conflict**, en cualquier motor.

PRUEBA EN EL MOTOR REAL

Que una consulta funcione en SQLite no garantiza que funcione igual en SQL Server o MySQL, y al revés. Si tu proyecto va a producción con un motor distinto al que usas para desarrollar, prueba tus consultas en ese motor. En nuestro proyecto se cambia con un perfil: `spring.profiles.active=mysql` o `sqlserver` (recuerda que SQL Server necesita antes `CREATE DATABASE crud_demo;`).

CAPÍTULO 08

EL ORM Y LOS JOIN

> En el código Java casi no escribimos SQL: lo genera Hibernate, el ORM que usa el proyecto. Pero los JOIN siguen ahí, y entenderlos es la diferencia entre usar el ORM y sufrirlo.

¿Qué ORM usamos?

La dependencia `spring-boot-starter-data-jpa` trae tres piezas que trabajan juntas:

Pieza	Qué hace
Spring Data JPA	Nos da los repositorios: `findAll`, `save`, los métodos derivados y `@Query`.
JPA (Jakarta Persistence)	La especificación: define `@Entity`, `@ManyToOne`, `@ManyToMany` y el lenguaje JPQL.
Hibernate (7.4)	El ORM propiamente dicho: implementa JPA y traduce JPQL al SQL de cada motor.
Dialecto	`SQLServerDialect`, `MySQLDialect` o `SQLiteDialect` (este último viene de `hibernate-community-dialects`).

JOIN en JPQL: con clases, no con tablas

JPQL se parece a SQL, pero habla de entidades y atributos de Java. Como JPA ya conoce la relación por `@JoinColumn`, no hace falta escribir el `ON`:

JPQL · ProductoRepository.buscarPorNombreDeCategoria

```
SELECT p FROM Producto p
JOIN p.categoria c
WHERE LOWER(c.nombre) = LOWER(:nombre)
ORDER BY p.precio
```

SQL que genera Hibernate (resumido) · el mismo en los tres motores

```
SELECT p.* FROM productos p
JOIN categorias c ON c.id = p.categoria_id
WHERE LOWER(c.nombre) = LOWER(?)
ORDER BY p.precio
```

El parámetro `:nombre` se envía aparte (el `?`), lo que protege contra la inyección SQL. Incluso un método derivado como `findByFavoritosId(Long id)` genera un JOIN con la tabla intermedia sin que escribamos nada.

El problema N+1 y el JOIN FETCH

Los favoritos son `LAZY`: se leen de la base solo cuando se usan. Al listar clientes en `/api/clientes/listar`, Hibernate hace 1 consulta para los clientes y luego 1 más por cada cliente para traer sus favoritos: **1 + N**

consultas. Con 5 clientes no se nota; con 10 000, sí. La solución es pedirle a Hibernate que haga el JOIN de una vez:

JPQL · ClienteRepository.listarConFavoritos

```
SELECT DISTINCT cl FROM Cliente cl
LEFT JOIN FETCH cl.favoritos p
LEFT JOIN FETCH p.categoria
ORDER BY cl.id
```

Con `spring.jpa.show-sql=true` puedes contar los `SELECT` en la consola: `/listar` hace varias consultas y `/con-favoritos` solo una.

ORM contra SQL directo

Criterio	ORM (JPA + Hibernate)	SQL directo
Cantidad de código	Poco: métodos y JPQL	Más: SQL y convertir filas en objetos
Cambiar de motor	El dialecto lo adapta	Puede haber que reescribir consultas
Control	Hibernate decide el SQL final	Total
Rendimiento	Bueno; cuidado con N+1	El mejor, si se escribe bien
Reportes complejos	JPQL tiene límites	Toda la potencia del motor

EL ORM NO TE LIBRA DEL SQL

El ORM nos ahorra trabajo, no conocimiento. Para leer lo que genera Hibernate, detectar un N+1, escribir un reporte que JPQL no alcanza o revisar datos directamente en la base, hay que saber SQL, y los JOIN son el corazón de ese SQL.

✓ PRÁCTICA **MANOS A LA OBRA**

1. Escribe una consulta que muestre las categorías que no tienen productos.
   Comprueba: debe devolver solo Ropa.
2. Escribe una consulta que muestre los clientes que no tienen favoritos.
   Comprueba: debe devolver solo a Jose Martinez.
3. Cuenta cuántos clientes tienen cada producto como favorito, incluidos los que no tiene nadie.
   Comprueba: Laptop 2, Clean Code 2, Producto sin categoria 0.
4. Muestra todos los productos y todas las categorías con un FULL OUTER JOIN en SQL Server o SQLite, y con LEFT + UNION + RIGHT en MySQL.
   Comprueba: 8 filas en los tres motores.
5. Ejecuta la consulta `WHERE c.nombre = 'deportes'` en los tres motores y explica con tus palabras por qué SQLite da un resultado distinto.

RESUMEN

Concepto	Recuerda
Normalización	Separar los datos en tablas para no repetirlos; se conectan con claves foráneas.
JOIN	Empareja las filas de dos tablas con la regla del `ON` (normalmente FK = PK).
INNER JOIN	Solo las filas con pareja. Los `NULL` nunca encuentran pareja.
LEFT JOIN	Todo lo de la izquierda; lo que falta de la derecha llega como `NULL`. Con `IS NULL` encuentra lo que falta.
FULL OUTER JOIN	Todo de los dos lados. SQL Server y SQLite (3.39+) sí; MySQL lo simula con UNION.
Tabla intermedia	Resuelve las relaciones muchos a muchos; se necesitan dos JOIN para atravesarla.
GROUP BY	Junta filas en grupos; con LEFT JOIN cuenta con `COUNT(columna)`, no con `COUNT(*)`.
Tres motores	Los JOIN son iguales; cambian `TOP` / `LIMIT`, los alias con espacios, las mayúsculas y las claves foráneas de SQLite.
ORM	Hibernate (JPA) con Spring Data JPA. JPQL usa clases y atributos; el dialecto lo traduce a cada motor.
N+1	Muchas consultas pequeñas por las relaciones LAZY; se evita con `JOIN FETCH`.

REFERENCIAS

Microsoft. *FROM clause plus JOIN, APPLY, PIVOT (Transact-SQL)*. learn.microsoft.com/sql/t-sql/queries/from-transact-sql

Oracle. *MySQL 8.4 Reference Manual: JOIN Clause*. dev.mysql.com/doc/refman/8.4/en/join.html

SQLite. *SELECT* (join-operator) y *SQLite Foreign Key Support*. sqlite.org/lang_select.html · sqlite.org/foreignkeys.html

Hibernate. *Hibernate ORM Documentation*. hibernate.org/orm/documentation

Spring. *Spring Data JPA Reference Documentation*. docs.spring.io/spring-data/jpa/reference

MARVIN RAFAEL MEZA PINEDA

Facilitador · Fundamentos de Desarrollo Backend con Java

Centro Nacional de Innovación y Tecnología Francisco "El Chele" Moreno · Temporada XIV · 2026